CMPSC448 Midterm: Identifying LLMs from Their Responses

Katherine Thornber

Although Large Language Models (LLMs) aim to answer the questions it is prompted for in a truthful manner, the stylistic choices in their answers vary across different models. While this is most obvious in the formatting of different LLMs, the wording and response structure may also vary in a patterned way such that one, with enough familiarity with the differences, could identify a LLM by its response. In this paper, we will investigate if neural networks can identify LLMs by their prompt and response. The task is a three-class classification problem involving responses labelled from ChatGPT, Claude, and Gemini evaluated with two neural network architectures: a convolutional neural network (CNN) and a recurrent neural network (RNN) implemented as a long short-term memory network (LSTM). From here, two research questions are investigated. The first simply asks if the generating LLM can be identified from its response without any further information. The second looks at the impact of the user's prompt, evaluating the models on prompt-only, response-only, and combined prompt-and-response inputs. Its classification is then evaluated through loss and a confusion matrix. An additional experiment was run to see if increasing the training budget and maximum response length improves performance.

This project uses the publicly available TXD-22 benchmark dataset, which does not have significant information on its collection process. However, the dataset has overwhelming benefits, in that it has a large amount of public data to pull and train from, and it is labelled appropriately enough for this experiment. The dataset contains 66,000 examples across 22 source categories, with columns corresponding to the prompt, response, and the LLM who wrote it. For this experiment, only three sources were extracted. This results in 2,987 shared prompts with 8,961 responses. The sources were then numerically encoded (ChatGPT = 0, Claude = 1, Gemini = 2), and the data is cleaned to remove missing values, surrounding whitespace, empty rows, and duplicate entries. The prompts were divided into 70% training, 15% validation, and 15% testing using a set seed (448) to retain reproducibility. Splitting the unique prompts instead of the data overall

allows for no overlap of prompts between the train and testing split, though similar questions may still be in both.

For neural network analysis, the text needs to be converted from full sentences to a tokenized system with a retained vocabulary list. The text is lowercased and tokenized using a regular expression that separates words and punctuation, but no further filtering was used. The vocabulary constructs itself from training prompts and responses, with a limit of 20,000 ordinary tokens. Special tokens were designated to represent padding and unknown words. The text is converted into integer token IDs, of which are numbered by first encounter, truncated to allow for faster processing, and padded on the right to ensure all inputs can be batched together. Prompts were limited to 64 tokens and responses to 256 tokens, and combined inputs can contain up to 64 prompt tokens, one separator token, and 256 response tokens.

Both models learn from 64-dimensional token embeddings. These embeddings are not pretrained to keep simplicity and consistency across the implementation. The CNN, which uses a one-dimensional convolution layer, slides across the text with a kernel width of three to detect local patterns. Then, ReLU introduces nonlinearity by setting negative activations to zero, global max-pooling retains the strongest activation from each filter, and the linear classification layer converts these features into three class scores, selecting the highest one for the prediction. LSTM embeds the tokens in the same way, then processes these vectors in order, using three primary gates to forget, input, and output from memory. It then produces a 64-dimensional representation of the sequence as a hidden state at the last real token, then the linear classification layer converts the representation into three class scores, and an argmax function chooses the highest one as the prediction. Both models return raw class scores for cross-entropy loss rather than applying a separate SoftMax operation.

Each architecture is trained separately for each of the three input modes, producing six experiments for each configuration of epochs and response length. The training follows a cycle of prediction, calculate error, and adjusting accordingly. For every batch of up to 64 examples, the CNN or LSTM first predicts by producing three class scores for each example

in the batch. Scores are then compared with the correct labels using PyTorch's cross-entropy loss function and produces the batch's average loss. Backpropagating the loss then determines the impact of each weight on the loss, and Adam uses those gradients with a learning rate of 0.001 to adjust the weights towards lower loss. The validation uses cross-entropy to measure performance but does not adjust the weights as its purpose is to check how well the learned weights work on separate examples. Training batches are shuffled, but validation and testing batches are not.

The initial configuration uses five epochs to train the model. After each epoch, validation loss is calculated, and the model weights with the lowest validation loss are retained. These weights are then evaluated on the test data set, measuring accuracy and macro-F1 score. Confusion matrices, as well as per-class precision, recall, and F1 scores provide additional results in training and allows for a detailed understanding of which classes were clearer than others. The follow-up configuration increases the epochs to ten and the response limit to 512 tokens, significantly reducing the number of responses needed to be truncated for processing. The seed and data splits remain unchanged. For compatibility on a MacBook laptop, the training was executed on a CPU using Python 3.12, PyTorch 2.14, pandas 3.0, and scikit-learn 1.9.

5 Epochs, Response Limit = 256 Tokens

Model, Mode	**Accuracy**	**Macro-F1**
CNN, Output	0.887	0.887
CNN, Input	0.333	0.299
CNN, Combined	0.891	0.891
LSTM, Output	0.634	0.627
LSTM, Input	0.333	0.254
LSTM, Combined	0.659	0.659

10 Epochs, Response Limit = 512 Tokens

Model, Mode	**Accuracy**	**Macro-F1**
CNN, Output	0.913	0.913
CNN, Input	0.333	0.321
CNN, Combined	0.915	0.915
LSTM, Output	0.831	0.321
LSTM, Input	0.333	0.262
LSTM, Combined	0.773	0.775

Table 1. Primary scores of the models, separated by resource allocations.
Output = Response only, Input = Prompt only, Combined = Prompt and Response.

The first research question only covers the results of the models with the mode Output. The initial response-only CNN achieves an accuracy of 88.34% accuracy and 0.8841 macro-F1, while the LSTM achieves 61.77% accuracy and 0.6110 macro-F1. The CNN's performance is substantially better than LSTM under these initial settings, supporting the conclusion that while both models exceeded the 33.33% chance baseline, CNN performs far better in these conditions. It is also worth mentioning that both the CNN and LSTM performed better with a larger resource allocation, where LSTM improved significantly and reached accuracies like the lower-resource CNN, although its low Macro-F1 score implies it still struggles to perform.

The second research question asks about prompt-only and combined inputs. Both prompt-only classifiers only achieved the baseline probability in its averages, which correlates with the dataset having three responses from three different LLMs per individual prompt. Since the prompts are identical to each other, there is no information for the LLM to gather. Combined inputs produce 88.79% accuracy for the CNN and 63.25% for the LSTM, which both show slight improvements from the response-only model.

Model	**Mode**	**Accuracy**	**Macro-F1**	**Test Loss**	**Best Epoch**	**Training Seconds**	**Truncated Train**	**Truncated Valid**	**Truncated Test**
CNN	Output	0.887	0.886	0.29	5	20.44	0.772	0.769	0.763
CNN	Input	0.333	0.229	1.1	4	6.71	0	0	0
CNN	Combo	0.891	0.891	0.30	5	24.25	0.772	0.769	0.763
LSTM	Output	0.634	0.627	0.81	3	34.45	0.772	0.769	0.763
LSTM	Input	0.333	0.254	1.1	5	9.46	0	0	0
LSTM	Combo	0.659	0.659	0.75	3	41.14	0.772	0.769	0.763

Table 2. Full scores of the models, with five epochs and a response limit of 256 tokens.

Model	Mode	Accuracy	Macro-F1	Test Loss	Best Epoch	Training Seconds	Truncated Train	Truncated Valid	Truncated Test
CNN	Output	0.913	0.913	0.23	10	75.03	0.015	0.015	0.019
CNN	Input	0.333	0.321	1.1	10	13.58	0	0	0
CNN	Combo	0.915	0.915	0.22	9	83.26	0.015	0.015	0.019
LSTM	Output	0.831	0.321	0.48	10	126.92	0.015	0.015	0.019
LSTM	Input	0.333	0.262	1.1	7	18.92	0	0	0
LSTM	Combo	0.773	0.775	0.58	10	143.03	0.015	0.015	0.019

Table 3. Full scores of the models, with ten epochs and a response limit of 512 tokens.

Due to hardware constraints, it was mandatory to keep the training limited and truncate the response tokens so that the time to train was within reason for this experiment. However, after seeing the first limitations truncated many of the responses, as well as the best epochs often being the last one, implying that the training for these models were primarily restricted by these two factors. Hence, an additional experiment was run to see if a larger resource allocation would positively impact the model's ability. Both models scored higher on the larger resource allocation runs, but the LSTM improved significantly. This could imply that LSTMs are more resource-heavy than CNNs.

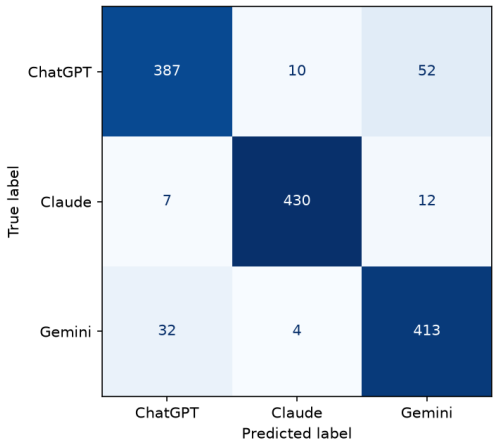


Figure 1. Confusion matrix for CNN-Output, 10/512

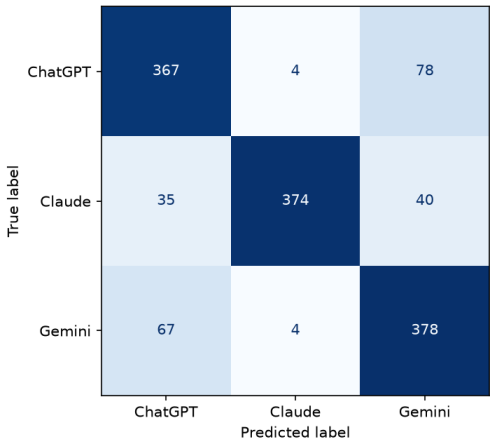


Figure 2. Confusion matrix for LSTM-Output, 10/512

These confusion matrices provide insight onto which LLMs are identifiable from others. Claude is the most consistently identifiable LLM by its response. The models have a harder time differentiating Gemini from ChatGPT, which could be explained by the similar layout in the outputs as well as the language each LLM uses. In all, the models were both able to identify the correct LLM with relatively high accuracy.

This study is limited to three LLMs derived from one public, undocumented dataset, so the findings may not be generalizable to other LLMs in the dataset or other datasets constructing the same idea. Since LLMs generate their responses individually, a prompt given to one LLM may result in different responses depending on prior context and conversational settings, which were removed in the dataset for this project. Truncation also limits the information available to classifiers, as the 256-token limit resulted in approximately 77% of the responses being truncated, thus the models were reliant on the beginning phrases of the response rather than the entire response. Each configuration was only run once on a single seed, which promotes replicability but does not assert generalizability. The follow-up experiment which reduced these limitations show slight improvement in both models.

This project demonstrates the importance of balanced data and prompt-based splitting when evaluating performance. Implementing each model brought a hands-on learning experience on the working layers and how they come together to create a language-processing neural network. Debugging the code and writing the essay clarified in detail how the embeddings, padding, convolution, recurrent memory, and gradient-based optimization worked together. In all, this project was very informative about the two models as well as how the three LLMs respond. Comparing accuracy, macro-F1 scores, and confusion matrices allowed for in-depth analysis of these models and how their different methods of learning vary. Another lesson from this project was that low accuracy does not necessarily indicate an implementation failure, as the prompt-only rate of 33% is explainable by the data given for the model. Additionally, accuracy and macro-F1 score are not always aligned, such as how the LSTM can be relatively accurate while retaining a low F1 score due to its confusion between two LLMs while consistently identifying the third. Finally, the CNN's stronger performance on a lower resource allocation shows that different

neural networks can be used when considering different hardware constraints and the task they are expected to perform, allowing for both models to be viable in different situations rather than one fully replacing the other.

To identify the LLM responsible for a response given a prompt, this project used two neural networks, CNN and LSTM, to examine the prompts, responses, and combination of both and classify the origin based on its learning network. In the initial experiments, both models exceeded the base chance of 33% when identifying LLMs based on their responses, which improved when combining prompt and responses. Due to the nature of the dataset, the models could only achieve the base chance when looking at prompts. This implies that source-identification is possible through the responses within the selected dataset. The CNN model performed substantially better than the LSTM model under the initial settings, while the LSTM improved with more resource allocation. Further experiments could assess consistency across random seeds and involve more LLMs to identify from to promote generalizability.